

ContextObjects: A Context-Centric, Retrieval-Aware Data Model that Coexists with Domain Standards for Real-Time Operations and GenAI

Francesc Mateu Amengual

MongoDB

francesc.mateu@mongodb.com

Abstract—Modern operational and AI systems repeatedly suffer from the same structural weakness: they reconstruct context on demand. When an abnormal lab result appears, a suspicious transaction occurs, a fleet vehicle emits critical signals or a customer journey crosses a threshold, systems must query multiple services and databases, re-join events and entities, and infer “what situation are we in?” before acting. This reconstruction logic is duplicated across services, analytics and GenAI workflows, increasing latency, cost and inconsistency.

We propose ContextObjects, a context-centric and retrieval-aware data model that persists “situations” as first-class data objects. A ContextObject represents a bounded situation (episode of care, fraud investigation, vehicle trip, retail journey) over time; it aggregates events and states from multiple systems, attaches semantic structure and enriches them with governance, provenance and AI-native artefacts such as summaries and embeddings.

ContextObjects are defined using SemanticObjects, an archetype-like modelling layer that is reference-model-agnostic but can be derived from and bound to domain standards such as openEHR archetypes, FHIR profiles, SNOMED CT concepts, retail open data models (e.g. MACH-style initiatives) and automotive specifications (e.g. COVESA VSS/VDM). Unlike these standards—which rightly optimise for capture and exchange—ContextObjects are explicitly storage- and retrieval-aware: their shape, grouping and indexing are designed so that applications and agents can retrieve the relevant situation in a small number of low-latency operations, while keeping semantically related data together and minimising total cost of ownership.

We further introduce ObjectMaps, declarative mappings from ingested objects into ContextObjects, and ContextViews, read-optimised projections of ContextObjects for specific access patterns. For AI workloads, ContextObjects become the primary unit of retrieval and reasoning. We define Con2L (Contexted Conversational Language), a structured JSON schema that LLM-based agents use to request context (intent, context type, subject, time window, focus, purpose), which is compiled into an internal ConQL (Context Query Language) over ContextObjects and executed on existing databases and vector search engines.

Our contributions are: (1) a context-centric, retrieval-aware data model that wraps existing standards; (2) a practical stack (SemanticObjects, ContextObjects, ObjectMaps, ContextViews, Con2L, ConQL) for implementing it; and (3) guidance on lifecycle, governance and organisational adoption. Our aim is not to replace openEHR, FHIR, SNOMED CT or sector-specific models, but to wrap and connect them inside a context-centric, retrieval-aware layer that supports real-time operations and GenAI with consistent semantics and governance.

Index Terms—context modeling, semantic objects, temporal reasoning, data governance, GenAI, LLMs, query languages

I. MOTIVATION

Many high-stakes decisions are inherently contextual:

- In healthcare, whether an abnormal lab result is concerning depends on the episode of care, diagnosis history and ongoing treatments.
- In fraud and risk, the meaning of a transaction depends on behavioural history, device fingerprints, geolocation and previous investigations.
- In automotive, the severity of a abnormal signal depends on the trip context, environment and recent events.
- In retail and CX, the meaning of a click or cart change depends on the entire customer journey and campaign context.

Today, most architectures reconstruct this context on demand: microservices and dashboards issue multiple queries to relational, document and streaming systems; orchestration layers stitch together partial information from APIs; data pipelines rebuild “episode views”, “case views” or “journey views” for analytics; RAG pipelines retrieve isolated documents or records without an explicit notion of a bounded situation.

This leads to:

- **latency and cost**, as every decision requires multiple lookups and joins;
- **duplicated logic**, as each team re-implements context reconstruction in its own layer;
- **governance gaps**, because policies are applied at record or table level, not at the level of “situations” that decisions and AI agents actually reason about;
- **fragile GenAI behaviour**, as LLMs see ad hoc subsets of data with incomplete or inconsistent context.

At the same time, most modelling traditions—including sophisticated ones like openEHR and FHIR—are intentionally capture- and exchange-first: openEHR’s Reference Model and archetypes focus on expressing clinical semantics safely and consistently over time; FHIR focuses on resource-based interoperability and API design; sector models such as MACH-style retail data models or COVESA’s vehicle models formalise domain entities and signals.

They are deliberately conservative about physical storage layout and indexing. As a result, many organisations operate at least three conceptual models: (1) an operational capture

model (EHR, core banking, telematics, commerce back end), (2) an analytics model (warehouse/lake schemas, cubes), and (3) an AI indexing model (text/vector indexes for RAG).

We ask a different question: what if context itself—episodes, cases, trips, journeys—were persisted as first-class, retrieval-aware data objects, constructed from existing standards but shaped for how applications and agents actually read?

II. RELATED WORK

We situate ContextObjects within existing traditions without attempting an exhaustive survey.

openEHR and dual modelling. openEHR separates a generic Reference Model from domain-specific archetypes and templates, and its COMPOSITION + EHR_CONTEXT structures already embody a notion of clinical situations. AQL provides a path-based query language over archetyped data. We adopt the spirit of dual modelling and path-like semantics, but generalise beyond a single reference model and domain.

FHIR and resource-centric APIs. FHIR defines resource structures and REST APIs for healthcare interoperability. It is optimised for exchange and client-server interaction, not for situation-level, cross-resource read models. We treat FHIR resources as source data and embedded payloads inside ContextObjects, not as something to replace.

Terminologies (SNOMED CT, LOINC, etc.). Clinical terminologies provide concept systems increasingly combined with NLP and embeddings. SemanticObjects bind to these terminologies; ContextObjects can integrate terminology-aware embeddings and queries, but terminologies remain the semantic backbone.

Open data models in other industries. The MACH Alliance’s open data model efforts in retail and COVESA’s VSS/VDM in automotive define standardised structures for products, orders and vehicle signals. We treat them as authoritative domain models whose instances are mapped into ContextObjects.

Read models, CQRS, data products and RAG. CQRS read models and data products are query-optimised projections of events or source databases. RAG and vector search index documents or chunks for semantic retrieval. ContextObjects can be seen as a specialised class of read models/data products whose boundary is a situation, and whose content is enriched with semantics, governance and embeddings, designed from the outset for both operations and GenAI.

III. THE CONTEXTOBJECTS STACK

The ContextObjects stack comprises four core elements and two supporting concepts:

- 1) **SemanticObjects** – archetype-like semantic definitions of reusable information structures.
- 2) **ContextObjects** – instances of context-type SemanticObjects representing bounded situations.
- 3) **ConQL (Context Query Language)** – an internal logical query model over ContextObjects.
- 4) **Con2L (Contexted Conversational Language)** – a JSON schema for agent requests for context.

- 5) **ObjectMaps** – declarative mappings from ingested objects into ContextObjects.
- 6) **ContextViews** – optional read-optimised projections of ContextObjects for specific access patterns.

Only ContextObjects and Con2L are intended to be visible at the application and agent level. SemanticObjects, ConQL, ObjectMaps and ContextViews support modelling and execution.

IV. SEMANTICOBJECTS: ARCHETYPE-LIKE SEMANTIC DEFINITIONS

A. Concept and Structure

SemanticObjects are formal definitions of reusable semantic structures such as LabResult, CardTransaction, VehicleEvent or CartSnapshot. They resemble openEHR archetypes conceptually but are reference-model-agnostic, explicitly binding-oriented to multiple underlying models, and intended to be mostly tool-generated from existing artefacts.

A SemanticObject includes:

- an identifier and version, e.g. `LabResult.v1`;
- a tree of SemanticNodes, each with:
 - `role`: field — group — section — event — event_series;
 - `dataType`: string — number — integer — boolean — date — datetime — object — array — code — quantity — reference;
 - optional constraints (cardinality, value ranges, patterns);
 - optional `terminologyBindings` (e.g. SNOMED CT, LOINC, ICD);
 - optional `structuralBindings` (e.g. an openEHR path, a FHIRPath, a VSS signal path).

SemanticObjects thus define what information means and how it relates to standards, without prescribing storage details.

B. Derivation and Tooling Assumptions

To avoid a heavy modelling burden, we assume most SemanticObjects are derived automatically from existing artefacts: openEHR archetypes and templates, FHIR profiles, sector data models, existing JSON schemas. Derivation produces a starting SemanticObject, which domain experts may refine.

Round-tripping between SemanticObjects and source artefacts is an important area for tooling, rather than a manual process. SemanticObjects are not intended to be “yet another hand-crafted meta-model”, but a thin, tool-managed semantic overlay.

V. CONTEXTOBJECTS: FIRST-CLASS SITUATION RECORDS

A. Definition

A ContextObject is an instance of a context-type SemanticObject (e.g. `OncologyEpisodeContext`, `FraudInvestigationContext`, `VehicleTripContext`). It represents a bounded situation defined by:

- one or more subjects (patient, account, vehicle, customer);
- a time interval over which the situation is considered;

- a collection of events and states, each an instance of a `SemanticObject`;
- optional native payloads from underlying standards;
- a governance slice derived from policies;
- provenance metadata;
- one or more embeddings.

The `ContextObject` is the primary unit of retrieval for many operations and AI tasks.

B. Anchors and Temporal Frame

Each `ContextObject` contains subjects with roles and identifiers:

```
"subjects": [
  { "role": "patient",
    "ehrId": "EHR:123",
    "fhirRef": "Patient/123" }
]
```

and a temporal frame:

```
"interval": {
  "start": "2024-01-01T09:00:00Z",
  "end": "2024-06-30T18:00:00Z"
}
```

C. Events and States

Events and states are stored as `SemanticObject` instances:

```
"events": [{
  "semanticObjectId": "LabResult.v1",
  "effectiveTime": "2024-02-10T10:00:00Z",
  "data": {
    "testName": "Glucose",
    "resultValue": 8.1,
    "unit": "mmol/L"
  },
  "bindings": {
    "openEHR": {
      "compositionUid": "comp-1",
      "path": "/content[openEHR-EHR-
        OBSERVATION.laboratory-test.v1]"
    },
    "FHIR": {
      "resourceType": "Observation",
      "id": "obs-123"
    }
  }
}]
```

D. Native Payloads

`ContextObjects` may embed or reference native payloads:

```
"nativePayloads": {
  "openEHR": {
    "compositions": [/* COMPOSITION JSON */]
  },
  "fhir": {
    "resources": [/* FHIR resources */]
  },
  "retail": {
    "orders": [/* commerce entities */]
  },
  "automotive": {
    "signals": [/* VSS/VDM signals */]
  }
}
```

E. Governance, Provenance and Embeddings

A governance slice captures local policy:

```
"governance": {
  "sensitivity": ["PHI"],
  "allowedPurposes": ["care_delivery"],
  "consentRefs": ["consent-001"]
}
```

Provenance documents origin:

```
"provenance": {
  "sourceSystems": ["EHR", "LIS", "FHIR_SERVER"],
  "projectionVersion": "v1",
  "lastUpdatedAt": "2024-06-30T18:05:00Z"
}
```

Embeddings enable semantic retrieval:

```
"embeddings": [
  { "type": "summary",
    "vector": [0.1, 0.2, "..."] },
  { "type": "risk_profile",
    "vector": [0.4, 0.5, "..."] }
]
```

VI. STORAGE- AND RETRIEVAL-AWARE MODELLING

Standards such as openEHR, FHIR and sector data models are intentionally neutral about storage layout and indexing strategies. Their remit is expressiveness and correctness of domain data, semantic stability and evolution, and interoperability and exchange.

They do not prescribe how data should be grouped for fast reads, what should be co-located in a single document or partition, or which indexes should be maintained for common situation-level queries.

`ContextObjects` are explicitly designed as retrieval-aware companions to these standards:

- The envelope (subjects, interval, governance, summary metrics, embeddings) is shaped to be index-friendly in document-oriented stores.
- The internal grouping of events and states keeps logically related information physically near one another.
- The design anticipates AI and agent read patterns: for many use cases, the context needed to take a decision or generate an explanation can be provided in one or a small number of reads.

We do not assume that every query always loads an entire `ContextObject`. In practice, many workloads will interact with `ContextViews` or partial slices tuned to specific latency and payload requirements.

VII. OBJECTMAPS AND CONTEXTVIEWS

A. ObjectMaps: From Capture Models to Contexts

`ObjectMaps` are declarative specifications that describe how ingested objects populate `ContextObjects`. They connect capture-oriented models to context-centric models without modifying the former.

Examples:

- In healthcare, an `ObjectMap` for `OncologyEpisodeContext` specifies which openEHR compositions and FHIR

resources correspond to diagnosis, treatment line changes, lab results and imaging summaries.

- In fraud, an `ObjectMap` for `FraudInvestigationContext` specifies which transaction events, login events and alerts map to which `SemanticObjects`.
- In automotive, an `ObjectMap` for `VehicleTripContext` specifies which VSS/VDM signals constitute trip start/stop, anomalies and key events.

`ObjectMaps` drive projector logic: when new events arrive, projectors consult `ObjectMaps` to determine which `ContextObjects` to update and how.

B. ContextViews: Read-Optimised Projections

`ContextViews` are optional, read-optimised projections derived from `ContextObjects`, shaped for specific access patterns. They contain a subset of fields from one or more `ContextObjects`, are relatively small and homogeneous, and support dedicated index configurations and SLOs.

Examples:

- **LabAlertView**: per context, minimal lab-related subset for alerting.
- **HighRiskTripView**: per trip, compressed anomaly and risk metrics.
- **PaymentDecisionView**: per investigation, condensed fraud-relevant subset.

Both `ContextObjects` and `ContextViews` are maintained by the same projection infrastructure.

VIII. CONTEXT BOUNDARIES, LIFECYCLE AND COMPACTION

Defining where one context starts and ends is domain-specific and non-trivial: clinical episodes may be split by lines of therapy; fraud investigations may merge multiple accounts; trips may be nested; retail journeys may overlap across channels.

`ContextObjects` do not prescribe universal boundary rules. Instead, each `ContextType` includes boundary definitions, expressed in terms of events, subject identifiers and temporal criteria. Projection logic applies these rules to assign events to contexts.

Over time, a single subject may accumulate many contexts. To avoid explosion and storage bloat:

- Historical contexts can be archived or compacted.
- Only a small number of contexts per subject may remain “active”.
- `ContextViews` can implement rolling-window strategies.

Lifecycle and compaction policies are part of the design of each `ContextType`.

IX. QUERYING CONTEXT: CONQL AND CON2L

A. ConQL: Internal Context Query Language

The Context Query Language (ConQL) is an internal logical representation of queries over `ContextObjects`. It provides constructs for:

- selecting context types;
- filtering by subjects and roles;

- temporal predicates (BEFORE, AFTER, DURING, OVERLAPS);
- structural predicates similar to CONTAINS;
- governance and quality predicates.

ConQL is not directly exposed to end users. It is generated by Con2L compilers, programmatic APIs, or graphical query builders.

Initial implementations need not be sophisticated optimisers; a pragmatic starting point is a rule-based translator that maps ConQL constructs to document database queries, vector search operations, and governance checks.

B. Con2L: Contexted Conversational Language

Con2L (Contexted Conversational Language) is a JSON-based schema that LLM-based agents use to describe what context they need.

Example (healthcare):

```
{
  "intent": "summarize",
  "contextTypes": ["OncologyEpisodeContext.v1"],
  "subject": {
    "role": "patient",
    "id": "EHR:123"
  },
  "timeWindow": { "lastMonths": 6 },
  "focus": [{
    "semanticObjectId": "LabResult.v1",
    "filters": {
      "terminology": "LOINC",
      "code": "12345-6",
      "minValue": 7.0
    }
  }],
  "constraints": {
    "purpose": "care_delivery",
    "allowedSensitivity": ["PHI"],
    "minDataQuality": "validated"
  },
  "output": {
    "style": "short_clinical_summary",
    "audience": "oncologist"
  }
}
```

Example (automotive):

```
{
  "intent": "investigate",
  "contextTypes": ["VehicleTripContext.v1"],
  "subject": {
    "role": "vehicle",
    "id": "VIN:WVWZZZ1JZXW000001"
  },
  "timeWindow": {
    "from": "2025-10-10T08:00:00Z",
    "to": "2025-10-10T09:00:00Z"
  },
  "focus": [{
    "semanticObjectId": "VehicleEvent.v1",
    "filters": {
      "signalPath": "Vehicle.Powertrain.Speed",
      "greaterThan": 120
    }
  }],
  "constraints": {
    "purpose": "fleet_operations",
    "disallowPIIInOutput": true
  },
}
```

```

"output": {
  "style": "explanation_with_evidence"
}

```

Execution pipeline:

- 1) Con2L is validated against its schema.
- 2) Governance constraints are checked.
- 3) A ConQL query is constructed from Con2L.
- 4) ConQL is translated into database + vector queries.
- 5) Returned context fragments are passed to the agent.

Agents thus operate at the level of contexts, not at the level of individual rows, resources or documents.

X. REAL-TIME MAINTENANCE AND CONSISTENCY

ContextObjects and ContextViews are maintained by projection processes that consume domain events, change data capture streams, and potentially batch backfills.

For each incoming event, a projector:

- 1) Uses ObjectMaps to determine which SemanticObject(s) to instantiate.
- 2) Determines which ContextObject(s) are affected.
- 3) Applies updates (append/modify events, update interval, recompute indicators).
- 4) Updates relevant ContextViews.

Consistency model. Systems of record remain the ultimate source of truth. ContextObjects are eventually consistent projections: they may lag underlying systems by seconds, but once updated, they provide a simple, stable view of the situation.

This trade-off matches many monitoring, alerting and decisioning use cases, where small delays are acceptable but repeated reconstruction is not.

XI. GOVERNANCE AND POLICY INTEGRATION

Governance is split into policy definition and policy evaluation:

Policy definition: Enterprise catalog and policy tools define data domains and classifications, sensitivity categories, allowed purposes and roles, and consent and contractual constraints.

Policy evaluation into governance slices: A policy engine evaluates these policies for ContextObject types and instances, producing governance slices persisted within each ContextObject.

Con2L and ConQL enforce these slices at query time, rejecting or modifying requests when requested purpose or role is incompatible with allowedPurposes and sensitivity, and enabling richer auditing.

This design avoids duplicating policy logic while bringing enforcement close to the situations that decisions and AI agents operate on.

XII. ORGANISATIONAL ROLES

For adoption, it matters who owns what. A minimal role split:

Domain teams (e.g. oncology, fraud, fleet, retail): Own ContextTypes, curate domain SemanticObjects, define boundary and lifecycle rules.

Data platform/architecture team: Implements the ContextObject store and indexing, builds projection infrastructure, implements ConQL translation and Con2L endpoints, manages ContextViews.

Governance/catalog team: Owns central policies and catalog metadata, maintains the policy engine, collaborates on audits.

This separation mirrors common data-mesh-inspired patterns: domains own semantics and context definitions; platform owns infrastructure; governance owns policy.

XIII. ADOPTION SCOPE AND INITIAL PATTERNS

Initial deployments will likely focus on one or two domains where context reconstruction is painful and costly, and semantics and governance matter. Start with a small number of ContextTypes per domain, derive a limited set of SemanticObjects and ObjectMaps for key events, and implement a handful of ContextViews aligned with immediate use cases.

Subsequent iterations can expand to more ContextTypes in the same domain, add further domains once patterns stabilise, and refine SemanticObjects as tooling matures.

This staged approach keeps scope realistic while preserving architectural generality.

XIV. DIFFERENTIATION

ContextObjects are related to but distinct from existing paradigms:

vs openEHR: ContextObjects reuse archetype-like semantics and context ideas, but generalise across domains and standards, and explicitly optimise for retrieval, AI consumption and mixed-standard integration.

vs FHIR: FHIR remains the API and resource model for exchange. ContextObjects sit above, integrating FHIR resources with other data into situation-level read models.

vs CQRS read models and data products: ContextObjects are a special class of read models whose boundary is a situation, enriched with multi-standard semantics, governance slices and embeddings.

vs knowledge graphs: ContextObjects do not require graph persistence, but can link to graphs and ontologies. They offer a document-centric representation friendlier to operational and AI workloads.

vs traditional RAG/vector stores: Rather than indexing arbitrary text chunks, ContextObjects index situations with explicit anchors, temporal frames and semantics.

XV. MINIMAL PRACTICAL PATTERN

To ground the concept, consider a minimal OncologyEpisodeContext pattern:

- **Subjects:** patient (EHR ID, FHIR Patient ref).
- **Interval:** from first tumour diagnosis to latest event.
- **Events:** PrimaryDiagnosis, TreatmentLineEvent, Key-LabResult, optionally ImagingSummary.
- **Native payloads:** openEHR compositions or FHIR resources.

- **Governance:** sensitivity=[PHI], allowedPurposes=[care_delivery].
- **Embeddings:** one “episode summary” embedding.

From this, one might define a LabAlertView for real-time lab alerts, a TumourBoardView for multi-disciplinary team support, and a Con2L schema for episode summaries.

This is not prescriptive, but an example of a concrete pattern implementable with modest scope.

XVI. CONCLUSION AND FUTURE WORK

We have introduced ContextObjects as a way to treat context—episodes, cases, trips, journeys—as a first-class, persistent, retrieval-aware data object, rather than something reconstructed repeatedly from disparate records and services.

ContextObjects are defined by SemanticObjects that can be largely derived from existing standards, populated via ObjectMaps from capture models such as openEHR, FHIR and sector data models, may be partially exposed as ContextViews for performance-critical access patterns, queried logically via an internal ConQL (Context Query Language), and requested by agents via Con2L (Contexted Conversational Language).

Crucially, ContextObjects are designed to coexist with and wrap domain standards, not to displace them. They complement capture- and exchange-first models with a read-first model tuned to the needs of real-time operations, governance and GenAI.

Future work includes building derivation and round-tripping tools for SemanticObjects and ObjectMaps, designing reference architectures and pilots for specific ContextTypes, implementing initial ConQL/Con2L runtimes on concrete databases, and empirically evaluating the impact of ContextObjects on decision latency, AI answer quality, governance enforcement and total cost of ownership.

ContextObjects are not a new standard in competition with openEHR, FHIR, MACH or COVESA. They are a proposal for how to organise, store and query “what is going on” across standards in a form that both operational systems and GenAI agents can consume consistently, efficiently and safely.